

```

#define ANALOG_INPUT_PIN A0
#define SAMPLING_FREQUENCY 9600 // Hz
#define CUTOFF_FREQUENCY 500 // Hz

// Updated coefficients for a 2nd-order Butterworth low-pass filter (500
Hz cutoff at 9600 Hz sampling rate)
float a1 = -1.54312113, a2 = 0.629604;
float b0 = 0.02162072, b1 = 0.04324144, b2 = 0.02162072;

// Filter state variables
float x1 = 0, x2 = 0; // Previous input values
float y1 = 0, y2 = 0; // Previous output values

void setup() {
    Serial.begin(9600);
}

// Function to apply the IIR Butterworth filter
float butterworthFilter(float x0) {
    float y0 = b0 * x0 + b1 * x1 + b2 * x2 - a1 * y1 - a2 * y2;

    // Shift values for the next iteration
    x2 = x1;
    x1 = x0;
    y2 = y1;
    y1 = y0;

    return y0;
}

void loop() {
    static unsigned long lastSampleTime = 0;
    unsigned long currentTime = micros();

    if (currentTime - lastSampleTime >= (1000000 / SAMPLING_FREQUENCY)) {
        lastSampleTime = currentTime;

        // Read the raw EMG signal
        float rawEMG = analogRead(ANALOG_INPUT_PIN);
    }
}

```

```
// Normalize input (assuming 10-bit ADC: 0-1023 range)
float inputSignal = rawEMG / 1023.0;

// Apply the Butterworth low-pass filter
float filteredSignal = butterworthFilter(inputSignal);

// Output the filtered signal
Serial.println(filteredSignal * 1023.0); // Convert back to ADC
scale if needed
    }
}
```